CPS 843: Computer Vision

Assignment 1: Noise Reduction & Histogram Equalization

Student Name: Geethavan Rajakumar

Student ID: 501176667

Department: Department of Computer Science, Toronto Metropolitan University

Date: October 5, 2026

1. Part 1: Noise Reduction by Multi-Image Averaging

1.1 Theoretical Background & Mathematical Principles

When capturing static scenes across multiple frames, observable sensor degradation is accurately modeled by additive noise:

$$g_i(x, y) = f(x, y) + \eta_i(x, y)$$

where $f(x, y)$ represents the true uncorrupted image scene(real image) and $\eta_i(x, y)$ represents an independent and identically distributed (i.i.d.) zero-mean Gaussian noise component with variance σ^2. Because the noise possesses zero mean, the estimated value of the noise is zero. ($E[\eta_i] = 0$) and the frames are entirely independent (no dependence on neighboring frames), computing the pixel-wise mean:

$$\bar{g}(x, y) = \frac{1}{K}\sum_{i=1}^{K} g_i(x, y)$$

First, substituting the noisy image model $g_i(x, y) = f(x, y) + \eta_i(x, y)$ into the averaging equation gives:

$$\bar{g}(x, y) = \frac{1}{K}\sum_{i=1}^{K} (f(x, y) + \eta_i(x, y))$$

Using the linear properties of summation, we separate the static image term from the noise term:

$$\bar{g}(x, y) = \frac{1}{K}\sum_{i=1}^{K} f(x, y) + \frac{1}{K}\sum_{i=1}^{K} \eta_i(x, y)$$

Because the underlying scene $f(x, y)$ is static across all acquired frames, it acts as a constant with respect to frame index i. Summing a constant K times yields $K \cdot f(x, y)$:

$$\bar{g}(x, y) = \frac{1}{K}Kf(x, y) + \frac{1}{K}\sum_{i=1}^{K} \eta_i(x, y) = f(x, y) + \frac{1}{K}\sum_{i=1}^{K} \eta_i(x, y)$$

This proves that the true image $f(x, y)$ stays completely intact and unaffected by the averaging process.

Next, we evaluate how the noise variance behaves after averaging K independent frames. Taking the variance (Var) of the averaged image:

$$Var(\bar{g}(x, y)) = Var\left(f(x, y) + \frac{1}{K}\sum_{i=1}^{K} \eta_i(x, y)\right)$$

Since $f(x,y)$ is a fixed constant, it has zero variance and drops out. Pulling out the scaling factor $\frac{1}{K}$ (which squares when taken outside variance) yields:

$$Var(\bar{g}(x,y)) = \frac{1}{K^2} \cdot Var\left(\sum_{i=1}^{K} \eta_i(x,y)\right)$$

Because each noise frame is independent, the variance of the sum equals the sum of the individual variances ($\sum_{i=1}^{K} \sigma^2 = K \cdot \sigma^2$):

$$Var(\bar{g}(x,y)) = \frac{1}{K^2} \cdot Var\left(K \cdot \sigma^2\right) = \frac{\sigma^2}{K}$$

Finally, taking the square root of the variance gives the standard deviation ($\sigma_{\bar{g}}$), which represents the actual visual noise amplitude (graininess) perceived by the human eye:

$$\sigma_{\bar{g}} = \sqrt{\frac{\sigma^2}{K}} = \frac{\sigma}{\sqrt{K}}$$

1.2 Quantitative Discussion & Results Analysis

The experimental results validate this theoretical framework. As evaluated on the 8-bit grayscale camera image with zero-mean additive Gaussian noise ($\sigma^2 = 0.01$), increasing the frame accumulation count K from 1 to 100 produces significant visual and quantitative improvements with regards to the additive noise and the calculated MSE.

Visually, single-frame instances ($K = 1$) exhibit heavy graininess (noise), whereas multi-frame averages ($K = 50, 100$) restore fine edges and smooth surface textures that closely approximate the original image $f(x,y)$. Quantitatively, the calculated Mean Squared Error (MSE) exhibits a steep decay that perfectly matches the theoretical prediction curve ($MSE \approx \frac{\sigma^2}{K}$), demonstrating that temporal/spatial frame averaging effectively eliminates zero-mean sensor noise without blurring structural image features.

1.3 Part 1 Visuals

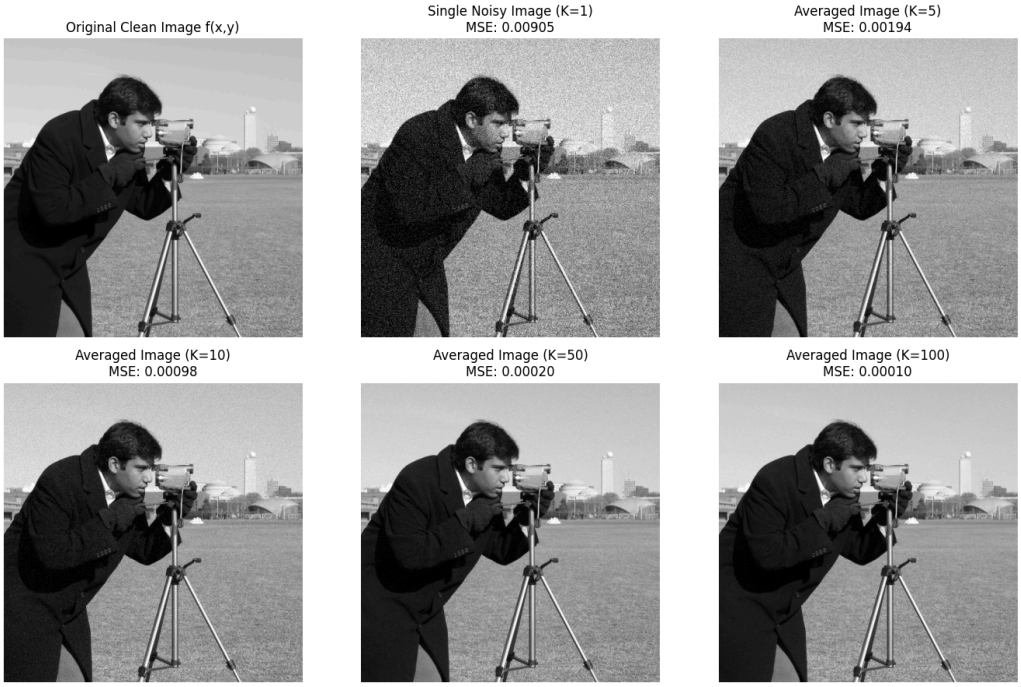


Figure 1: Multi-image averaging visual results across varying frame counts K.

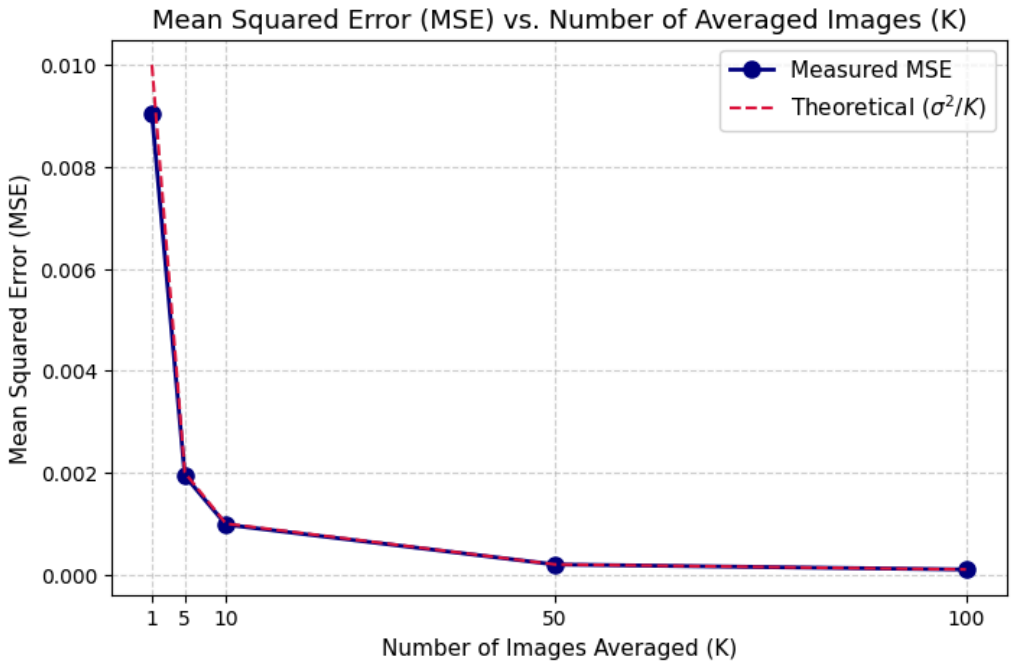


Figure 2: Quantitative Mean Squared Error (MSE) reduction as a function of frame count K.

2. Part 2: Global Histogram Equalization

2.1 Theoretical Background & Algorithm Architecture

Global histogram equalization is a contrast enhancement technique designed to stretch an image's brightness levels across the entire available dynamic range (0 to 255 for standard 8-bit grayscale images).

Instead of leaving pixels clumped together in a dark or dull gray range, the algorithm redistributes them evenly from pure black to pure white. This reveals hidden details and maximizes visual contrast.

The algorithm operates in three primary computational steps:

Step A (Calculate Pixel Probabilities): First, we count how often each gray level (r_k) appears in the image and divide by the total number of pixels ($M \times N$). This produces a Probability Density Function (PDF):

$$1. \quad p_r(r_k) = \frac{n_k}{M \times N}$$

which tells us the percentage of the image occupied by each brightness level:

Step B (Build the Cumulative Sum): Next, we calculate the Cumulative Distribution Function (CDF). By adding up the probabilities step-by-step from dark to light, $T(r_k)$ tracks the running total percentage of pixels up to brightness level r_k:

$$2. \quad T(r_k) = \sum_{j=0}^{k} p_r(r_j)$$

Step C (Map Pixels to New Values): Finally, we scale the running totals to the full [0:255] brightness range to create a Look-Up Table (LUT):

$$3. \quad s_k = round\Big((L - 1) \cdot T(r_k)\Big)$$

Instead of recalculating every pixel individually, the computer uses this lookup table to update the whole image at once. This stretches out the image's brightness levels evenly across the spectrum, unclustering pixel values and making hidden details and textures easy to see.

2.2 Visual & Histogram Analysis

Testing the algorithm on the low-contrast image (of the moon) shows a dramatic improvement in visual quality. The original picture looks dark and washed-out because almost all its pixels are squished into a narrow band of dark grays, burying fine surface details in shadow.

After applying histogram equalization, the pixel brightness is spread across the entire spectrum, from pure black (0) to pure white (255). As a result, hidden textures and surface details on the moon instantly pop out and become easy to see.

The histogram plots back up what we see in the images: while the original histogram shows a single tall spike where all the dark pixels are jammed together, the equalized histogram spreads those pixel counts across the whole [0:255] scale, proving that the overall contrast was successfully expanded.

2.3 Part 2 Visuals

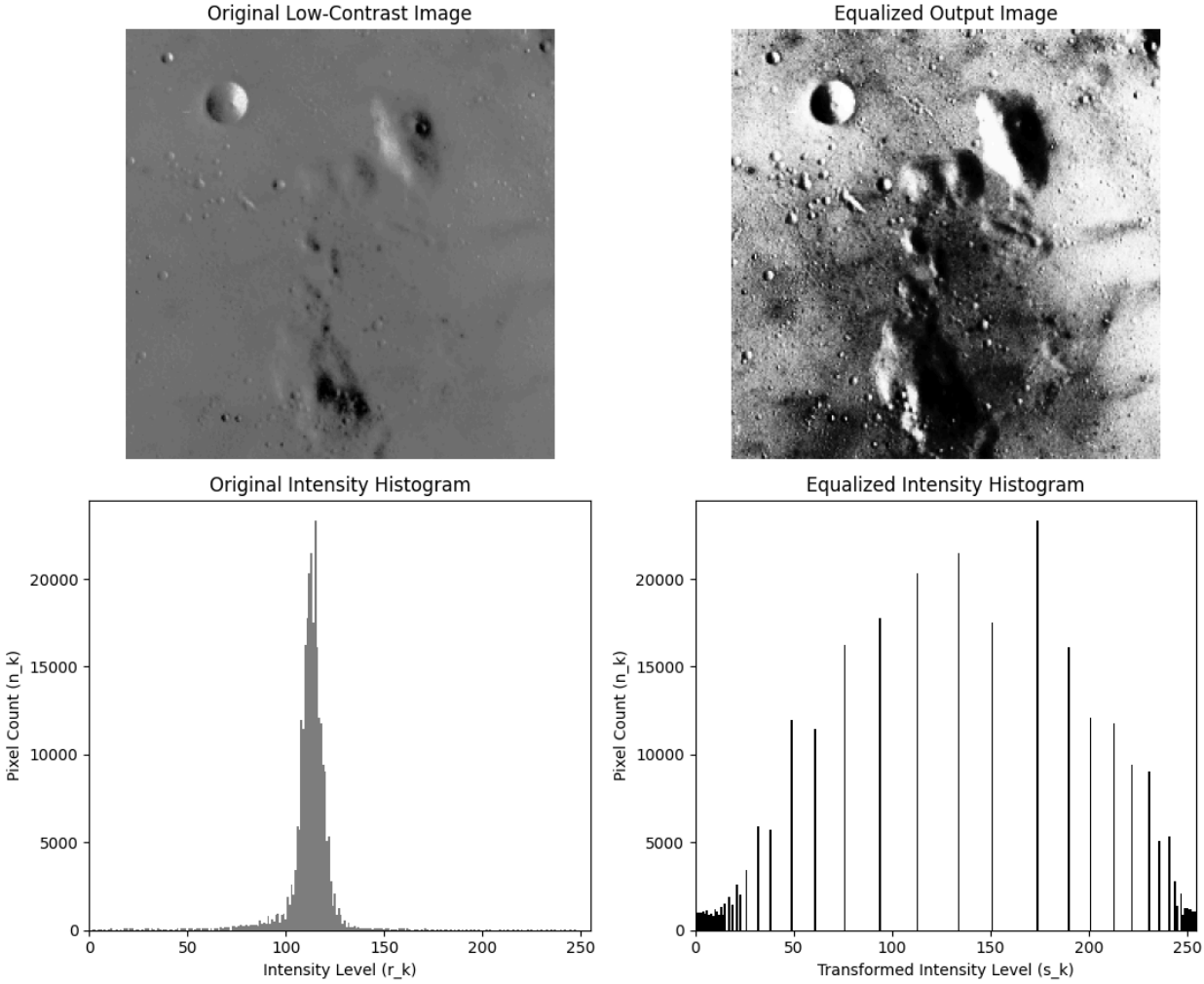


Figure 3: Global histogram equalization performance and intensity distribution comparison.

Appendix: Python Source Code

Python

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from skimage import data

# Enable inline plotting
%matplotlib inline

# Set random seed for exact reproducibility across runs
np.random.seed(42)

#
================================================================================
======
# PART 1: NOISE REDUCTION BY MULTI-IMAGE AVERAGING
#
================================================================================
======

# Load clean benchmark 8-bit image directly from skimage package
clean_uint8 = data.camera() # uint8 grayscale matrix [0, 255]

# Convert to float32 normalized to [0, 1] to prevent uint8 overflow/underflow clipping
f = clean_uint8.astype(np.float32) / 255.0

# Parameters
variance = 0.01
sigma = np.sqrt(variance)  # Standard deviation = 0.1
K_values = [1, 5, 10, 50, 100]

mse_dict = {}
averaged_images = {}

# Multi-Image Generation & Averaging Loop
for K in K_values:
    # Stack K noisy instances of shape (K, H, W)
    noisy_stack = np.zeros((K, f.shape[0], f.shape[1]), dtype=np.float32)

    for i in range(K):
        # Generate zero-mean Gaussian noise matrix
        eta = np.random.normal(loc=0.0, scale=sigma, size=f.shape).astype(np.float32)
```

```
        # Form noisy instance g_i = f + eta
        noisy_stack[i] = f + eta

    # Compute multi-frame mean g_bar across axis 0
    g_bar = np.mean(noisy_stack, axis=0)

    # Clip for display visualization
    g_bar_clipped = np.clip(g_bar, 0.0, 1.0)
    averaged_images[K] = g_bar_clipped

    # Compute Mean Squared Error (MSE) relative to clean reference image f
    mse = np.mean((f - g_bar_clipped) ** 2)
    mse_dict[K] = mse
    print(f"K = {K:3d} | Measured MSE = {mse:.6f}")

# Part 1 Visualization: Image Grid (Original + K Instances)
fig1, axes1 = plt.subplots(2, 3, figsize=(14, 9))

axes1[0, 0].imshow(f, cmap='gray', vmin=0, vmax=1)
axes1[0, 0].set_title("Original Clean Image f(x,y)")
axes1[0, 0].axis('off')

axes1[0, 1].imshow(averaged_images[1], cmap='gray', vmin=0, vmax=1)
axes1[0, 1].set_title(f"Single Noisy Image (K=1)\nMSE: {mse_dict[1]:.5f}")
axes1[0, 1].axis('off')

axes1[0, 2].imshow(averaged_images[5], cmap='gray', vmin=0, vmax=1)
axes1[0, 2].set_title(f"Averaged Image (K=5)\nMSE: {mse_dict[5]:.5f}")
axes1[0, 2].axis('off')

axes1[1, 0].imshow(averaged_images[10], cmap='gray', vmin=0, vmax=1)
axes1[1, 0].set_title(f"Averaged Image (K=10)\nMSE: {mse_dict[10]:.5f}")
axes1[1, 0].axis('off')

axes1[1, 1].imshow(averaged_images[50], cmap='gray', vmin=0, vmax=1)
axes1[1, 1].set_title(f"Averaged Image (K=50)\nMSE: {mse_dict[50]:.5f}")
axes1[1, 1].axis('off')

axes1[1, 2].imshow(averaged_images[100], cmap='gray', vmin=0, vmax=1)
axes1[1, 2].set_title(f"Averaged Image (K=100)\nMSE: {mse_dict[100]:.5f}")
axes1[1, 2].axis('off')

plt.tight_layout()
plt.show()
```

```
# Part 1 Visualization: Quantitative MSE vs K Plot
plt.figure(figsize=(8, 5))
plt.plot(K_values, [mse_dict[k] for k in K_values], 'o-', color='navy', linewidth=2, markersize=8,
label='Measured MSE')

# Plot theoretical curve MSE = sigma^2 / K
theoretical_mse = [variance / k for k in K_values]
plt.plot(K_values, theoretical_mse, '--', color='crimson', linewidth=1.5, label=r'Theoretical ($\sigma^2/K$)')

plt.title("Mean Squared Error (MSE) vs. Number of Averaged Images (K)", fontsize=13)
plt.xlabel("Number of Images Averaged (K)", fontsize=11)
plt.ylabel("Mean Squared Error (MSE)", fontsize=11)
plt.xticks(K_values)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=11)
plt.show()

#
==========================================================================
======
# PART 2: HISTOGRAM EQUALIZATION FROM SCRATCH
#
==========================================================================
======
print("--- RUNNING PART 2: HISTOGRAM EQUALIZATION ---")

# Load low-contrast benchmark test image directly from skimage
input_img = data.moon()  # uint8 low-contrast grayscale matrix
M, N = input_img.shape

# Step A: Compute normalized intensity histogram (PDF)
counts, _ = np.histogram(input_img.ravel(), bins=256, range=(0, 256))
pdf = counts / (M * N)

# Step B: Compute Cumulative Distribution Function (CDF) mapping function
cdf = np.cumsum(pdf)

# Step C: Direct Intensity Mapping using Look-Up Table (LUT) vectorization
L = 256
lut = np.round((L - 1) * cdf).astype(np.uint8)
equalized_img = lut[input_img]

# Compute bin counts for equalized output image for visual display
```

```
eq_counts, _ = np.histogram(equalized_img.ravel(), bins=256, range=(0, 256))

# Part 2 Visualization: Required 2x2 Subplot Grid
fig2, axes2 = plt.subplots(2, 2, figsize=(11, 9))

# Top-Left: Original Low-Contrast Image
axes2[0, 0].imshow(input_img, cmap='gray', vmin=0, vmax=255)
axes2[0, 0].set_title("Original Low-Contrast Image")
axes2[0, 0].axis('off')

# Top-Right: Equalized Output Image
axes2[0, 1].imshow(equalized_img, cmap='gray', vmin=0, vmax=255)
axes2[0, 1].set_title("Equalized Output Image")
axes2[0, 1].axis('off')

# Bottom-Left: Intensity Histogram of Original Image
axes2[1, 0].bar(range(256), counts, color='gray', width=1.0, edgecolor='none')
axes2[1, 0].set_title("Original Intensity Histogram")
axes2[1, 0].set_xlabel("Intensity Level (r_k)")
axes2[1, 0].set_ylabel("Pixel Count (n_k)")
axes2[1, 0].set_xlim([0, 255])

# Bottom-Right: Intensity Histogram of Equalized Image
axes2[1, 1].bar(range(256), eq_counts, color='black', width=1.0, edgecolor='none')
axes2[1, 1].set_title("Equalized Intensity Histogram")
axes2[1, 1].set_xlabel("Transformed Intensity Level (s_k)")
axes2[1, 1].set_ylabel("Pixel Count (n_k)")
axes2[1, 1].set_xlim([0, 255])

plt.tight_layout()
plt.show()
```